

Assignment 3

Submission requirements

Your work will not be graded if your notebook doesn't include output. In other words, **make sure to rerun your notebook before submitting to Gradescope** (Note: if you are using Google Colab: go to Edit > Notebook Settings and uncheck Omit code cell output when saving this notebook, otherwise the output is not printed).

Additional points may be deducted if these requirements are not met:

- Comment your code;
- Each graph should have a title, labels for each axis, and (if needed) a legend. Each graph should be understandable on its own;
- Try and minimize the use of the global namespace (meaning, keep things inside functions).

Import libraries

```
In [1]: import numpy as np
import pandas as pd
import seaborn as sns # for nicer plots
sns.set(style="darkgrid") # default style
from sklearn.model_selection import train_test_split
import tensorflow as tf
from matplotlib import pyplot as plt
import keras_tuner as kt
from keras_tuner import HyperParameters
```

This lab continues our study of linear regression. You'll train your first models with Tensorflow, using a real dataset to predict car prices from their features. Note that Tensorflow is a rapidly changing library. This means you'll often see warnings about deprecations. You can ignore the warnings in our labs.

Step 1: Data ingestion

You'll use the [Automobile Data Set](#) from 1985 Ward's Automotive Yearbook that is part of the [UCI Machine Learning Repository](#).

This dataset has been pre-downloaded and uploaded to bCourses alongside this notebook for your convenience.

```
In [2]: columns = [
    "symboling", "normalized-losses", "make", "fuel-type", "aspiration",
    "num-of-doors", "body-style", "drive-wheels", "engine-location",
    "wheel-base", "length", "width", "height", "curb-weight",
    "engine-type", "num-of-cylinders", "engine-size", "fuel-system",
    "bore", "stroke", "compression-ratio", "horsepower", "peak-rpm",
    "city-mpg", "highway-mpg", "price"
]

car_data_init = pd.read_csv(
    "./hw3_data/imports-85.data",
    header=None,
    names=columns,
    na_values="?"
)
# Display top five rows
print('Shape of data:', car_data_init.shape)
car_data_init.head()
```

Shape of data: (205, 26)

Out[2]:

	symboling	normalized-losses	make	fuel-type	aspiration	num-of-doors	body-style	drive-wheels	engine-location
0	3	NaN	alfa-romero	gas	std	two	convertible	rwd	front
1	3	NaN	alfa-romero	gas	std	two	convertible	rwd	front
2	1	NaN	alfa-romero	gas	std	two	hatchback	rwd	front
3	2	164.0	audi	gas	std	four	sedan	fwd	front
4	2	164.0	audi	gas	std	four	sedan	4wd	front

5 rows × 26 columns



Step 2: Data preprocessing

This step is essential for preparing the data in a format that is suitable for ML algorithms. It helps ensure data quality and improvements in model performance.

Exercise 1: Column selection (5 points)

To keep things simple, you will:

1. Retain only the following columns: ['horsepower', 'peak-rpm', 'city-mpg', 'highway-mpg', 'price']. Name the new dataframe `car_data`.
2. Display the data type of each column;
3. Convert the data type of each columns to numeric. Coerce missing values to NaN. Hint: use `pd.to_numeric()` method;
4. Display the data type of each column after the transformation performed at point 3.

```
In [3]: # YOUR CODE HERE
# 1. retain only certain cols
car_data = car_data_init[['horsepower', 'peak-rpm', 'city-mpg', 'highway-mpg', 'pri

# 2. display data type of each col
for col in car_data.columns:
    print('Datatype of ', col, car_data[col].dtype)
# 3. convert each col to numeric
car_data[col] = pd.to_numeric(car_data[col], errors='coerce')
# 4. display data type of each col
print(f'Datatype of {col} after converting:', car_data[col].dtype)
```

```
Datatype of  horsepower float64
Datatype of horsepower after converting: float64
Datatype of  peak-rpm float64
Datatype of peak-rpm after converting: float64
Datatype of  city-mpg int64
Datatype of city-mpg after converting: int64
Datatype of  highway-mpg int64
Datatype of highway-mpg after converting: int64
Datatype of  price float64
Datatype of price after converting: float64
```

Exercise 2: Example (row) selection (5 points)

To keep things simple again, you will:

1. Print the shape of the `car_data`;
2. Remove examples (rows) that have missing value(s). Note that in doing so, you will overwrite the `car_data` dataset. You should end up with 199 examples after this cleaning.
3. Print the shape of the `car_data` again.

It's important to acknowledge that there are multiple approaches to handling missing features, and simply discarding examples with any missing feature, though straightforward, may not be the most optimal solution. However, for the sake of simplicity, you will implement this strategy in this assignment.

```
In [4]: # YOUR CODE HERE
# 1. shape of car data
print('Shape of car data', car_data.shape)

# 2. drop rows with nans
```

```
car_data = car_data.dropna()

# 3. print shape
print('Shape of car data after dropping nans', car_data.shape)
```

Shape of car data (205, 5)

Shape of car data after dropping nans (199, 5)

```
In [5]: car_data.isna().sum()
```

```
Out[5]: horsepower      0
        peak-rpm        0
        city-mpg        0
        highway-mpg     0
        price           0
        dtype: int64
```

Exercise 3: Data shuffling (10 points)

Since you'll be using Batch Gradient Descent (BGD) for training, it is important that **each batch is a random sample of the data** so that the gradient computed is representative. Note that the original data (above) appears sorted by *make* in alphabetic order.

Using NumPy and Pandas methods:

1. Create a list of indices corresponding to the rows in the `car_data` dataset. Call this list *indices*. Print this list;
2. Shuffle *indices* using the `np.random.permutation()` method. Call the resulting array *shuffled_indices*. Print this array;
3. Use the method `dataframe.reindex()` to change the ordering of the `car_data` dataset based on the order in the *shuffled_indices* array. Note that in doing so, you will overwrite the original dataset. Print the top 5 rows.

```
In [6]: np.random.seed(0)
        # YOUR CODE HERE

        # 1. indices corresponding to rows of car_data dataset
        indices = list(car_data.index)
        print('List of indices', indices)

        # 2. shuffle indices
        shuffled_indices = np.random.permutation(indices)
        print('Shuffled indices', shuffled_indices)

        # 3. reindex car_data using shuffled indices
        car_data = car_data.reindex(shuffled_indices)
        car_data.head()
```

```
List of indices [0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63,
64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84,
85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104,
105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121,
122, 123, 124, 125, 126, 127, 128, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141,
142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158,
159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175,
176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192,
193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204]
Shuffled indices [ 19 175 109 95 182 189 5 145 13 166 64 127 170 151 83 7 3
4 135
38 77 165 150 48 164 63 125 184 190 100 47 17 58 86 113 23 194
141 4 143 199 159 69 27 137 152 66 8 78 101 172 74 202 89 99
155 25 31 116 41 59 136 188 186 20 158 142 57 168 54 115 144 92
93 126 149 187 111 15 28 104 121 49 200 110 65 2 62 179 132 46
11 192 76 197 183 173 128 96 114 163 196 53 0 97 112 98 67 147
42 72 52 51 88 14 156 24 191 140 21 16 81 107 55 103 79 3
119 169 203 6 71 87 124 160 176 161 94 177 12 122 105 36 60 68
1 123 167 43 108 138 204 18 39 139 56 162 134 35 29 117 157 32
185 133 181 33 148 174 153 30 102 85 82 118 154 201 75 80 26 171
84 193 180 195 40 61 146 91 73 90 37 22 10 106 70 198 120 50
178]
```

```
Out[6]:
```

	horsepower	peak-rpm	city-mpg	highway-mpg	price
19	70.0	5400.0	38	43	6295.0
175	92.0	4200.0	27	32	9988.0
109	97.0	5000.0	19	24	12440.0
95	69.0	5200.0	31	37	7799.0
182	52.0	4800.0	37	46	7775.0

Exercise 4: Define outcome and features (5 points)

Create two dataframes as follows:

1. The first dataframe contains our outcome of interest: ['price']. Note, this is what we are aiming to predict. Name this dataframe Y. Print shape of Y.
2. The second dataframe contains our features of interest: ['horsepower', 'peak-rpm', 'city-mpg', 'highway-mpg']. Name this dataframe X. Print shape of X.

```
In [7]: # YOUR CODE HERE
Y = car_data['price']
print('Shape of Y', Y.shape)

X = car_data[['horsepower', 'peak-rpm', 'city-mpg', 'highway-mpg']]
print('Shape of X', X.shape)
```

Shape of Y (199,)
 Shape of X (199, 4)

Exercise 5: Data splits (10 points)

Using the `train_test_split()` method available in scikit-learn:

1. Partition the (X, Y) data into training, validation, and test sets using a splitting rule of [60%, 20%, 20%], with a random state set to 1234. Name the resulting dataframes as follows: X_train, X_val, X_test, Y_train, Y_val, Y_test. Hint: To create these three partitions you will utilize the `train_test_split()` method twice (all the other arguments of the method are set to default values.). You should obtain [119, 40, 40] examples for training, validation, and test, respectively.
2. Print the shape of each dataframe.

Note: The validation set is crucial for evaluating different hyperparameter configurations and selecting those that yield optimal model performance. This approach avoids utilizing the test dataset during model training, as it is assumed to be "unknown" at that stage.

```
In [8]: # YOUR CODE HERE
# 1. train, test, val split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2)
X_train, X_val, Y_train, Y_val = train_test_split(X_train, Y_train, test_size=0.25)

# 2. shape of each dataframe
print('Shape of X_train', X_train.shape)
print('Shape of X_val', X_val.shape)
print('Shape of X_test', X_test.shape)
print('Shape of Y_train', Y_train.shape)
print('Shape of Y_val', Y_val.shape)
print('Shape of Y_test', Y_test.shape)
```

Shape of X_train (119, 4)
 Shape of X_val (40, 4)
 Shape of X_test (40, 4)
 Shape of Y_train (119,)
 Shape of Y_val (40,)
 Shape of Y_test (40,)

Exercise 6: Data standardization (10 points)

With this concept in mind, complete the following tasks:

1. Output the quantile values (0.25, 0.5, 0.75, 0.95) for all features in the X_train dataset. Are these values uniformly scaled across features?
2. Standardize all features in X_train, X_val, and X_test. Label the resulting dataframes as X_train_std, X_val_std, and X_test_std, respectively. Hint: standardize the validation and test data using the mean and standard deviation computed from the training data. Why?

- Similar to point 2. but now standardize the outcome variable. Label the resulting dataframes as Y_train_std, Y_val_std, and Y_test_std.

```
In [9]: # YOUR CODE HERE
# 1. quantile values for each feature in X_train
for feature in X_train.columns:
    print('Feature', feature)
    print('25th quantile', X_train[feature].quantile(q=0.25))
    print('50th quantile', X_train[feature].quantile(q=0.50))
    print('75th quantile', X_train[feature].quantile(q=0.75))
    print('95th quantile', X_train[feature].quantile(q=0.95))

# 2. standardize the features
X_train_std = (X_train - X_train.mean()) / X_train.std()
X_val_std = (X_val - X_train.mean()) / X_train.std()
X_test_std = (X_test - X_train.mean()) / X_train.std()

# 3. standardize target
Y_train_std = (Y_train - Y_train.mean()) / Y_train.std()
Y_val_std = (Y_val - Y_train.mean()) / Y_train.std()
Y_test_std = (Y_test - Y_train.mean()) / Y_train.std()
```

```
Feature horsepower
25th quantile 69.0
50th quantile 90.0
75th quantile 116.0
95th quantile 175.1
Feature peak-rpm
25th quantile 4800.0
50th quantile 5200.0
75th quantile 5500.0
95th quantile 5900.0
Feature city-mpg
25th quantile 20.5
50th quantile 25.0
75th quantile 31.0
95th quantile 37.099999999999994
Feature highway-mpg
25th quantile 25.5
50th quantile 30.0
75th quantile 37.0
95th quantile 42.099999999999994
```

Answers

- The values are not uniformly scaled across features, we can see that `peak-rpm` is in the 1000's, while `city-mpg` 25th quantile is 19 and 95th quantile is 37. Thus we must standardize the features such that parameter updates during learning are uniform.
- We use the mean and std from the train data to standardize the test and val datasets to prevent data leakage, as we cannot possibly know the exact mean and std of the test/val datasets to standardize them in practice, the point of these sets is to see how our trained model performs with just the training data's information.

Step 3: Exploratory data analysis (EDA)

EDA plays a very important role in ML. The goal here is to develop a good understanding of our training dataset, identify any data quality issues, understand patterns and relationships, which in turn, aids in subsequent modeling and interpretations.

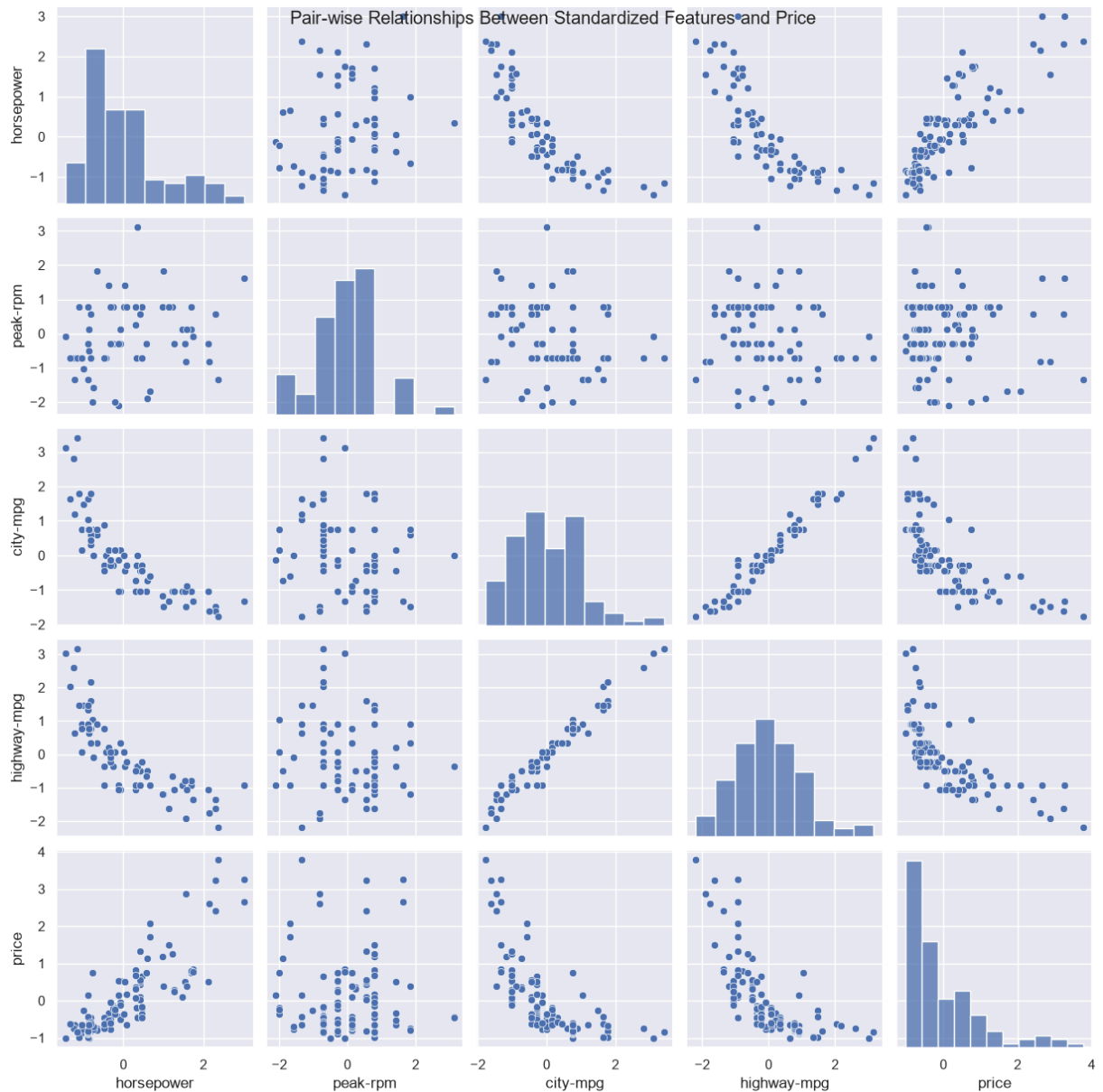
Exercise 7: Scatterplot matrix (10 points)

In this exercise you will use some simple yet useful techniques to visualize the distribution of the data.

Let's start with:

1. A scatterplot matrix to visualize the pair-wise correlations between different features and outcome in the (X_train_std, Y_train_std) data. You will use the `sns.pairplot()` method from the seaborn library imported at the top of the notebook;
2. Is any of the variables in the data normally distributed? Is it necessary for the explanatory or target variable to be normally distributed in order to train a ML model?

```
In [10]: # YOUR CODE HERE
# 1. scatterplot matrix of pari-wise correlations b/w features and target
pairplot_data = pd.concat([X_train_std, Y_train_std], axis=1)
pair_plot = sns.pairplot(pairplot_data)
pair_plot.fig.suptitle("Pair-wise Relationships Between Standardized Features and P
plt.show()
```



Answers

2. `peak-rpm`, `city-mpg`, and `highway-mpg` are slightly normally distributed, as you can see their histograms in the matrix roughly follow a bell curve. It is not necessary for the explanatory or target variables to be normally distributed for an ML model, but if you do have a variable that is very skewed, it can be helpful to transform the variable to be more similar to a normal distribution to help with learning.

Exercise 8: Correlation matrix (10 points)

In this exercise you will:

1. Plot a correlation matrix in the form of a heatmap to visualize the linear relationships between different features and outcome in the (X_train_std, Y_train_std) data. Hint: this

example here is very useful:

https://seaborn.pydata.org/examples/many_pairwise_correlations.html

2. Answer the following questions:

- Which two features are likely to be most redundant?
- Which feature is likely to be least useful for predicting price?

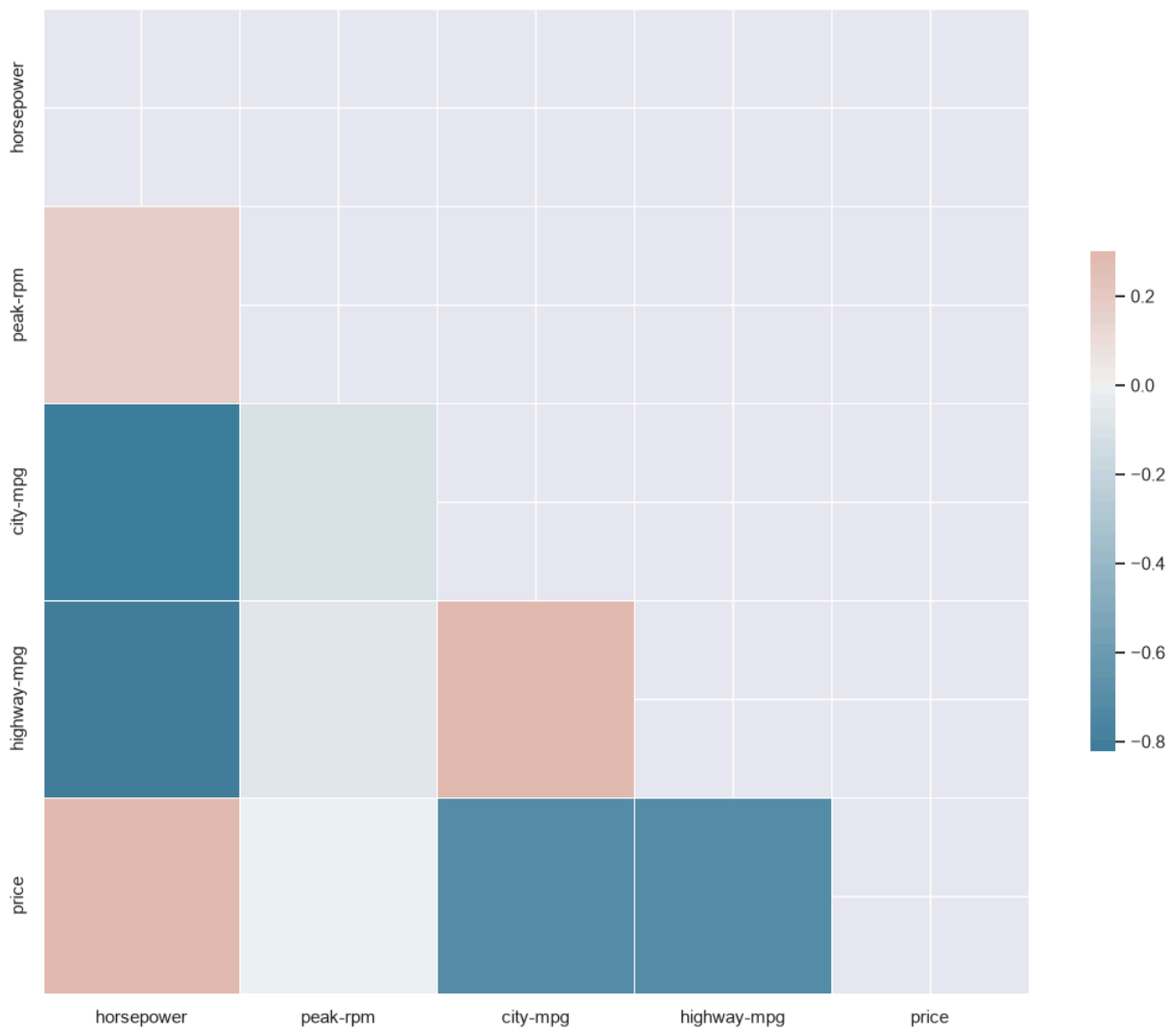
```
In [11]: # YOUR CODE HERE
# Compute the correlation matrix
corr = pairplot_data.corr()

# Generate a mask for the upper triangle
mask = np.triu(np.ones_like(corr, dtype=bool))

# Set up the matplotlib figure
f, ax = plt.subplots(figsize=(11, 9))

# Generate a custom diverging colormap
cmap = sns.diverging_palette(230, 20, as_cmap=True)

# Draw the heatmap with the mask and correct aspect ratio
sns.heatmap(corr, mask=mask, cmap=cmap, vmax=.3, center=0,
            square=True, linewidths=.5, cbar_kws={"shrink": .5})
plt.tight_layout()
plt.show()
```



Answers 2. The two features most likely to be redundant are `city-mpg` and `highway-mpg`, as they have essentially the same distribution and have nearly the exact same relationships with the other features.

The feature least likely to be useful for predicting price is `peak-rpm`, because it has basically no correlation with any other feature, and most importantly, `price`.

Step 4: Modeling

Exercise 9: Baseline model (5 points)

Let's start by evaluating a baseline model. Precisely, you'll use the average price of cars in the training set as our baseline model -- that is, the baseline always predicts the average price regardless of the input.

1. Implement this baseline using the `Y_train_std` data and print the average price. Note: You can revert the price variable to the original scale for interpretation purposes.

```
In [12]: # YOUR CODE HERE

# baseline pred = avg
baseline_pred = Y_train_std.mean()
print('Baseline Prediction - Average Price of Training Data Cars (Scaled)', baseline_pred)
print('Baseline Prediction - Average Price of Training Data Cars (Original Scale)',
```

```
Baseline Prediction - Average Price of Training Data Cars (Scaled) 3.265361837132813
5e-17
```

```
Baseline Prediction - Average Price of Training Data Cars (Original Scale) 12715.840
336134454
```

Exercise 10: Improvement over Baseline with TensorFlow (10 points)

Let's train a linear regression model much like we did in the previous assignment, but this time using TensorFlow.

1. Fill in the `NotImplemented` parts of the `build_model()` function below by following the instructions provided as comments. Hint: refer to the course webpage in [bCourses/Modules/Module 03/live session demo](#) for an example.
2. Build and compile a model using the `build_model()` function and the (`X_train_std`, `Y_train_std`) data. Set `learning_rate = 0.0001`. Call the resulting object `model_tf`.
3. Train `model_tf` using the (`X_train_std`, `Y_train_std`) data. Set `num_epochs = 5`. Pass the (`X_val_std`, `Y_val_std`) data for validation. Hint: see the documentation behind the `tf.keras.Model.fit()` method.
4. Generate a plot with the loss values on the y-axis and the epoch number on the x-axis for visualization. Make sure to include axes name and title. Hint: check what the `tf.keras.Model.fit()` method returns.

More notes on point 1: the idea is to build a *computational graph* for linear regression, and then send data through it. There are many ways to build graphs, but [TensorFlow Keras API](#) is recommended.

```
In [13]: def build_model(num_features, learning_rate):
        """Build a TF linear regression model using Keras.

        Args:
            num_features: The number of input features.
            learning_rate: The desired learning rate for SGD.
```

```

Returns:
    model: A tf.keras model (graph).
    """
    # This is not strictly necessary, but each time you build a model, TF adds
    # new nodes (rather than overwriting), so the colab session can end up
    # storing lots of copies of the graph when you only care about the most
    # recent. Also, as there is some randomness built into training with SGD,
    # setting a random seed ensures that results are the same on each identical
    # training run.
    tf.keras.backend.clear_session()
    tf.random.set_seed(0)

    # Build a model using keras.Sequential. While this is intended for neural
    # networks (which may have multiple layers), we want just a single layer for
    # linear regression.
    model = tf.keras.Sequential()
    model.add(tf.keras.layers.Dense(
        units=1,          # output dim
        input_shape=(num_features,), # input dim
        use_bias=True,     # use a bias (intercept) param
        kernel_initializer='ones', # initialize params to 1
        bias_initializer='ones',   # initialize bias to 1
    ))

    # We need to choose an optimizer. We'll use GD, which is actually mini-batch GD
    optimizer = tf.keras.optimizers.SGD(learning_rate=learning_rate)

    # Finally, compile the model. This finalizes the graph for training.
    # We specify the MSE loss and the optimizer above
    model.compile(loss='mse', optimizer=optimizer)

    return model

```

```

In [14]: tf.random.set_seed(0)
# 2. Build and compile model
# YOUR CODE HERE
print(X_train.shape)
model_tf = build_model(X_train.shape[1], learning_rate=0.0001)

# 3. Fit the model
# YOUR CODE HERE
history = model_tf.fit(
    x=X_train_std, y=Y_train_std,
    validation_data=[X_val_std, Y_val_std],
    epochs=5
)

# 4. train val loss curves
def plot_loss(history):
    plt.plot(history.history['loss'], label='train loss')
    plt.plot(history.history['val_loss'], label='test loss') # val_loss is the vali
    plt.xlabel('Epoch')
    plt.ylabel('MSE ("BMI")')
    plt.legend()
    plt.grid(True)

```

```
plot_loss(history)
```

```
(119, 4)
```

WARNING:tensorflow:From c:\Berkeley_DataSci\Lib\site-packages\keras\src\backend\common\global_state.py:82: The name tf.reset_default_graph is deprecated. Please use tf.compat.v1.reset_default_graph instead.

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Epoch 1/5

c:\Berkeley_DataSci\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

4/4 ————— 0s 81ms/step - loss: 5.8039 - val_loss: 5.3795

Epoch 2/5

4/4 ————— 0s 36ms/step - loss: 5.7883 - val_loss: 5.3667

Epoch 3/5

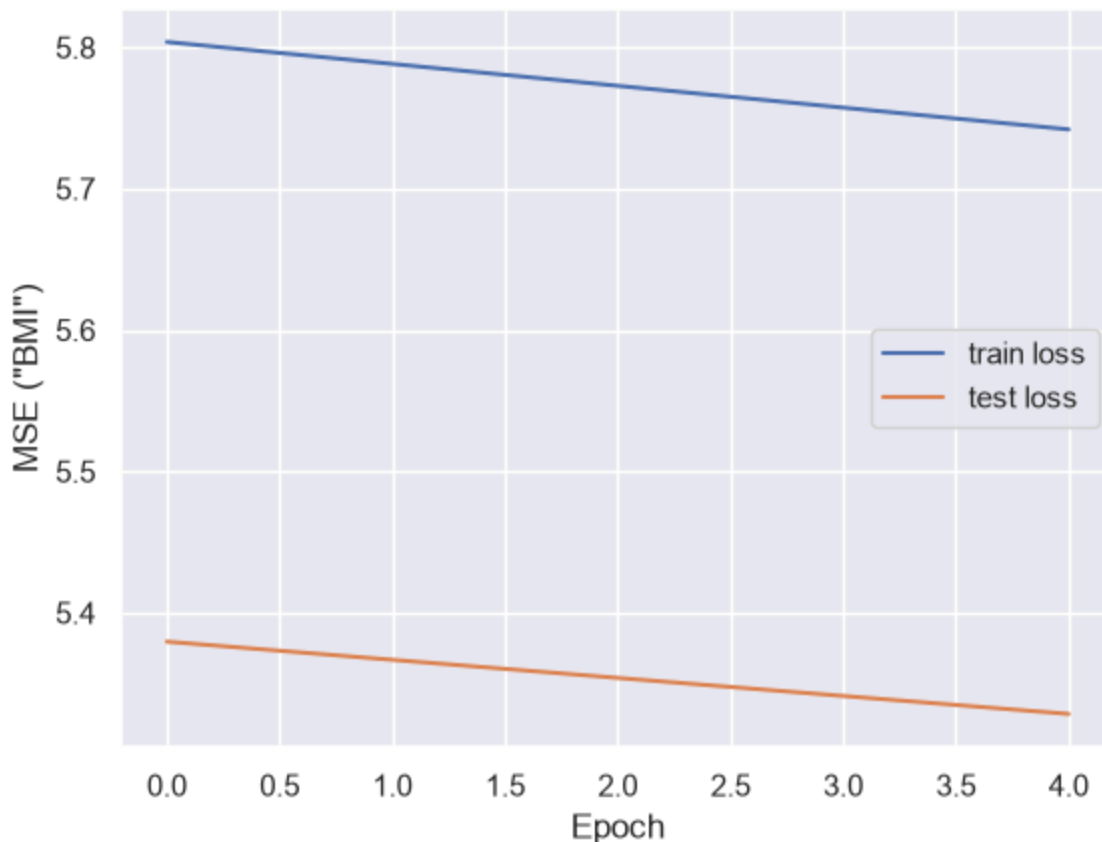
4/4 ————— 0s 37ms/step - loss: 5.7728 - val_loss: 5.3539

Epoch 4/5

4/4 ————— 0s 37ms/step - loss: 5.7574 - val_loss: 5.3411

Epoch 5/5

4/4 ————— 0s 26ms/step - loss: 5.7420 - val_loss: 5.3284



Step 5: Hyperparameter tuning

Hyperparameter tuning is a crucial step in optimizing ML models. It involves systematically adjusting hyperparameters such as learning rate, number of epochs, and optimizer to find the model configuration that leads to the best generalization performance.

This tuning process is typically conducted by monitoring the model's performance on the validation vs. training set. It's important to note that using the test set for hyperparameter tuning can compromise the integrity of the evaluation process by violating the assumption of "blindness" of the test data.

Exercise 11: Hyperparameter tuning (10 points)

1. Fine-tune the **learning rate** and **number of epochs** hyperparameters of *model_tf* to determine the setup that yields the most optimal generalization performance. Feel free to explore various values for these hyperparameters. Hint: you can manually test different hyperparameter values or you can use the [Keras Tuner](#). If you decide to work with the Keras Tuner, define a new model building function named `build_model_tuner()`.

After identifying your preferred model configuration, print the following information:

2. The learned parameters of the tuned model (this should include the bias term). Hint: use `[model_name].layers[0].get_weights()`.
3. The loss at the final epoch on both the training and validation datasets;
4. The difference between the last-epoch loss observed on the training and validation datasets.

Please note that we will consider 'optimal model configuration' any last-epoch training loss that is below 0.31 and any last epoch validation loss that is below 0.48. Hint: do not specify the `batch_size` argument in the `fit()` function (it defaults to 32).

```
In [21]: tf.random.set_seed(0)
# YOUR CODE HERE
def build_model_tuner(hp):
    tf.keras.backend.clear_session()

    model = tf.keras.Sequential([
        tf.keras.layers.Dense(
            units=1,
            input_shape=(X_train_std.shape[1],),
            use_bias=True,
            kernel_initializer="ones",
            bias_initializer="ones"
        )
    ])

    learning_rate = hp.Float(
        "learning_rate",
        min_value=0.00001,
```

```

        max_value=0.01,
        sampling="log"
    )

    model.compile(
        optimizer=tf.keras.optimizers.SGD(
            learning_rate=learning_rate
        ),
        loss="mse"
    )

    return model

# run tuner
tuner = kt.Hyperband(
    build_model_tuner,
    objective="val_loss",
    max_epochs=50,
    factor=3,
    hyperband_iterations=1,
    directory="keras_tuner_results",
    project_name="car_price_regression",
    overwrite=True
)

tuner.search(
    X_train_std,
    Y_train_std,
    validation_data=(X_val_std, Y_val_std),
    callbacks=[
        tf.keras.callbacks.EarlyStopping(monitor='val_loss',
                                          patience=3,
                                          restore_best_weights=True)
    ],
    verbose=1
)

```

Trial 90 Complete [00h 00m 10s]
val_loss: 1.921775221824646

Best val_loss So Far: 0.4676121771335602
Total elapsed time: 00h 05m 33s

In [22]: # 2, 3, 4

```

# 2. best model
best_model = tuner.get_best_models(num_models=1)[0]
best_hp = tuner.get_best_hyperparameters(num_trials=1)[0]
print("Best learning rate:", best_hp.get("learning_rate"))
print("Best epoch count:", best_hp.values.get("tuner/epochs"))
print("Learned parameters:", best_model.layers[0].get_weights())

# 3. 4. training and validation losses
best_trial = tuner.oracle.get_best_trials(num_trials=1)[0]

final_train_loss = best_trial.metrics.get_last_value("loss")

```

```

final_val_loss = best_trial.metrics.get_last_value("val_loss")

print("Final training loss:", final_train_loss)
print("Final validation loss:", final_val_loss)
print(
    "Difference between validation and training loss:",
    final_val_loss - final_train_loss
)

```

Best learning rate: 0.009201505727005656

Best epoch count: 50

Learned parameters: [array([[1.0887624],
 [-0.1668224],
 [0.16256933],
 [0.05721387]], dtype=float32), array([0.01344597], dtype=float32)]

Final training loss: 0.28639814257621765

Final validation loss: 0.4676121771335602

Difference between validation and training loss: 0.18121403455734253

c:\Berkeley_DataSci\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
 super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Step 6: Evaluation and Generalization

Now that you've determined the optimal set of hyperparameters, it's time to evaluate your optimized (tuned) model on the test data to gauge its performance in real-world scenarios, commonly known as inference.

Exercise 12: Computing MSE (10 points)

1. Calculate the MSE on both (X_train_std, Y_train_std) and (X_test_std, Y_test_std) datasets. Hint: You can utilize the `model.evaluate()` method provided by tf.keras.
2. Does the model demonstrate strong generalization capabilities? Provide an explanation based on your observations. Hint: compare train vs. test MSE.
3. Generate a plot to visualize the accuracy of the predictions. Plot the actual (observed) Y_test values on the x-axis and the predicted Y_test values on the y-axis. Additionally, include a 45-degree line in the plot for reference. Ensure that the plot contains appropriate axis labels and a title. Provide commentary on the model's fit based on this visualization. Hint: You can utilize the `model.predict()` method available in tf.keras.

In [23]: `# YOUR CODE HERE`

```

# 1. test MSE using optimized model
test_MSE = best_model.evaluate(X_test_std, Y_test_std)

```

```

train_MSE = best_model.evaluate(X_train_std, Y_train_std)

print('Train MSE', train_MSE)
print('Test MSE', test_MSE)

# 3. Y_test vs. Y_hat visual
y_pred = best_model.predict(X_test_std, verbose=0).flatten()
y_actual = Y_test_std.to_numpy()

# Plot actual versus predicted values
plt.figure(figsize=(8, 6))
plt.scatter(y_actual, y_pred, alpha=0.7, label="Predictions")

# 45-degree reference line
line_min = min(y_actual.min(), y_pred.min())
line_max = max(y_actual.max(), y_pred.max())
plt.plot(
    [line_min, line_max],
    [line_min, line_max],
    color="red",
    linestyle="--",
    label="Perfect prediction"
)

plt.xlabel("Actual Standardized Price")
plt.ylabel("Predicted Standardized Price")
plt.title("Actual vs. Predicted Car Prices")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

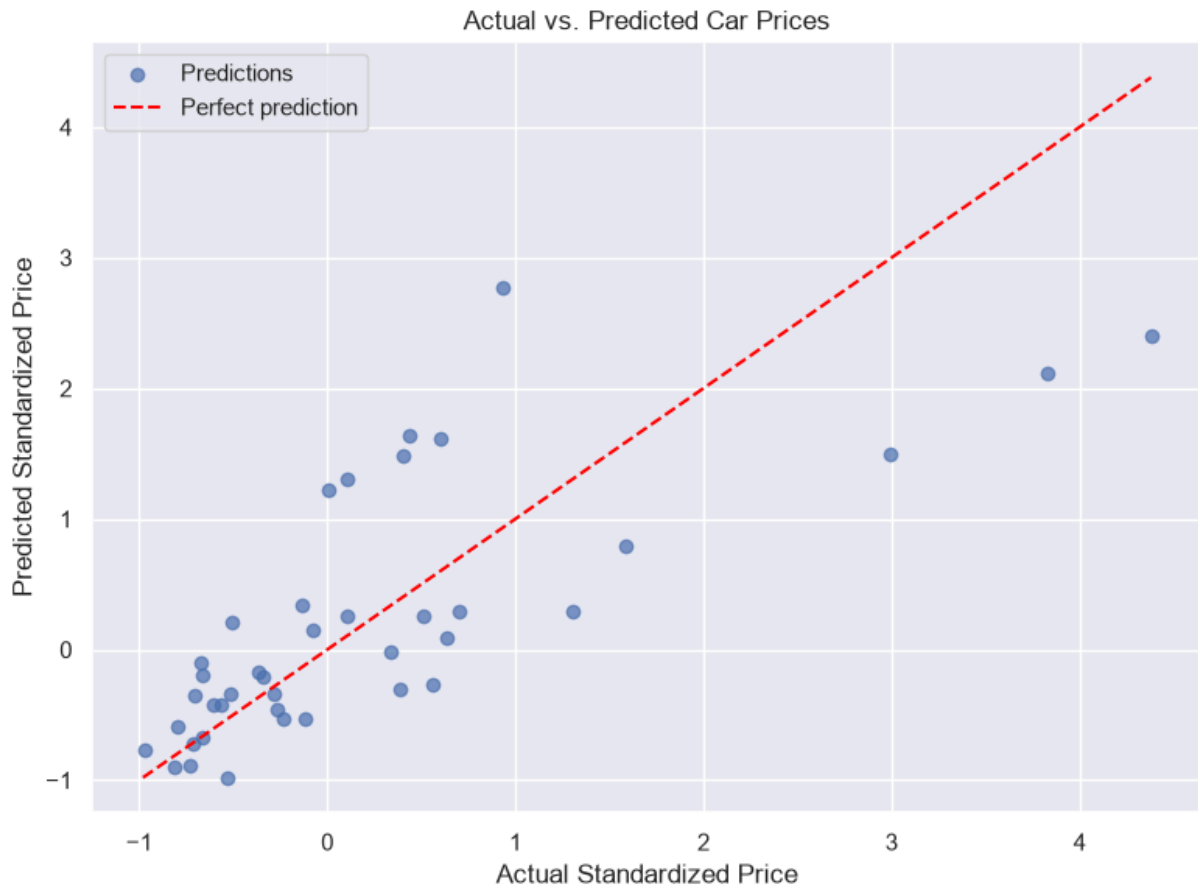
```

2/2 ————— 0s 28ms/step - loss: 0.6180

4/4 ————— 0s 9ms/step - loss: 0.2845

Train MSE 0.28447583317756653

Test MSE 0.6180120706558228



Answers 2. The model provides okay generalization, we see that the train 0.28 while the test MSE is 0.62. The gap between the 2 is 0.34, which isn't the worst, but that is over a 100% increase from the train MSE, so generalization could be better.

3. The model seems to have good predictions when the standardized price of the car is low. We can see this by looking at the bottom left of the visual: the dots are clustered around the perfect-fit line. It does worse when the price of the car is higher, as we can see as we look to the right of the graph, the dots get further and further from the line.

Additional practice (not graded)

In Exercise 12, you reported an aggregated MSE. Let's revisit the exercise by:

1. Performing a subgroup evaluation of the model. Specifically, calculate the test data MSE for the following makes: ['alfa-romero', 'audi', 'chevrolet', 'dodge', 'honda'].
2. Addressing the question: Is the model "fair" across each make?

In []: `# YOUR CODE HERE`